OP-TEE 성능 최적화 실전 기법

이 문서는 OP-TEE 시스템의 성능을 최적화하기 위한 실전 기법들을 상세히 설명합니다. 실제 벤치마킹 결과와 함께 구체적인 최적화 코드 예제를 제공합니다.

1. 메모리 관리 최적화

1.1 메모리 풀 최적화

문제: 빈번한 malloc/free로 인한 성능 저하 및 메모리 단편화

해결책: 사전 할당된 메모리 풀 사용

```
// 최적화된 메모리 풀 구현
struct memory_pool {
void *base_addr;
size_t total_size;
size_t block_size;
size_t num_blocks;
uint8_t *free_bitmap;
struct mutex pool_mutex;
uint32_t allocation_count;
uint32_t peak_usage;
};
```

```
// 메모리 풀 초기화
static TEE_Result init_memory_pool(struct memory_pool *pool,
size_t  block_size, size_t  num_blocks)
{
size_t total_size = block_size * num_blocks;
size_t bitmap_size = (num_blocks + 7) / 8; // 비트맵 크기
/*
* 1단계: 메모리 할당
*/
pool->base_addr = malloc(total_size);
if (!pool->base_addr) {
EMSG("Failed to allocate pool memory");
return TEE_ERROR_OUT_OF_MEMORY;
}
pool->free_bitmap = malloc(bitmap_size);
if (!pool->free_bitmap) {
free(pool->base_addr);
return TEE_ERROR_OUT_OF_MEMORY;
}
/*
* 2단계: 풀 구조체 초기화
*/
pool->total_size = total_size;
```

```
pool->block_size = block_size;
pool->num_blocks = num_blocks;
pool->allocation_count = 0;
pool->peak_usage = 0;
/*
* 3단계: 비트맵 초기화 (모든 블록 자유 상태)
*/
memset(pool->free_bitmap, 0, bitmap_size);
/*
* 4단계: 뮤텍스 초기화
*/
mutex_init(&pool->pool_mutex);
DMSG("Memory pool initialized: %zu blocks of %zu bytes",
num_blocks, block_size);
return TEE_SUCCESS;
}


// 빠른 블록 할당
static void *pool_alloc_fast(struct memory_pool *pool)
{
void *ptr = NULL;
size_t block_index = 0;
mutex_lock(&pool->pool_mutex);
```

```
/*
* 1단계: 자유 블록 검색 (비트맵 스캔)
*/
for (size_t i = 0; i < (pool->num_blocks + 7) / 8; i++) {
if (pool->free_bitmap[i] != 0xFF) { // 자유 블록이 있는 바이트
/*
* 2단계: 바이트 내에서 자유 비트 찾기
*/
for (int bit = 0; bit < 8; bit++) {
if (!(pool->free_bitmap[i] & (1 << bit))) {
block_index = i * 8 + bit;
if (block_index < pool->num_blocks) {
/*
* 3단계: 블록 할당 표시
*/
pool->free_bitmap[i] |= (1 << bit);
pool->allocation_count++;
if (pool->allocation_count > pool->peak_usage) {
pool->peak_usage = pool->allocation_count;
}
/*
* 4단계: 블록 주소 계산
*/
```

```
ptr = (char *)pool->base_addr +
(block_index * pool->block_size);
goto out;
}
}
}
}
}
out:
mutex_unlock(&pool->pool_mutex);
return ptr;
}


// 빠른 블록 해제
static void pool_free_fast(struct memory_pool *pool, void *ptr)
{
if (!ptr) return;
/*
* 1단계: 블록 인덱스 계산
*/
uintptr_t offset = (uintptr_t)ptr - (uintptr_t)pool->base_addr;
size_t block_index = offset / pool->block_size;
if (block_index >= pool->num_blocks) {
```

```
EMSG("Invalid pointer for pool free: %p", ptr);
return;
}
mutex_lock(&pool->pool_mutex);
/*
* 2단계: 비트맵에서 블록 해제 표시
*/
size_t byte_index = block_index / 8;
int bit_index = block_index % 8;
if (pool->free_bitmap[byte_index] & (1 << bit_index)) {
pool->free_bitmap[byte_index] &= ~(1 << bit_index);
pool->allocation_count--;
/*
* 3단계: 보안을 위한 메모리 지우기
*/
memzero_explicit(ptr, pool->block_size);
} else {
EMSG("Double free detected: %p", ptr);
}
mutex_unlock(&pool->pool_mutex);
}


// 메모리 풀 사용 예제
```

```
static struct memory_pool small_pool; // 256바이트 블록
static struct memory_pool medium_pool; // 1KB 블록
static struct memory_pool large_pool; // 4KB 블록


static TEE_Result init_optimized_allocator(void)
{
TEE_Result res = TEE_SUCCESS;
/*
* 다양한 크기의 메모리 풀 초기화
*/
res = init_memory_pool(&small_pool, 256, 1000);
if (res != TEE_SUCCESS) return res;
res = init_memory_pool(&medium_pool, 1024, 500);
if (res != TEE_SUCCESS) return res;
res = init_memory_pool(&large_pool, 4096, 100);
if (res != TEE_SUCCESS) return res;
return TEE_SUCCESS;
}


// 크기별 최적 풀 선택
static void *optimized_alloc(size_t size)
{
if (size <= 256) {
```

```
return pool_alloc_fast(&small_pool);
} else if (size <= 1024) {
return pool_alloc_fast(&medium_pool);
} else if (size <= 4096) {
return pool_alloc_fast(&large_pool);
} else {
// 큰 할당은 일반 malloc 사용
return malloc(size);
}
}
```

1.2 캐시 친화적 데이터 구조

```
// 캐시 라인 크기 정의
#define CACHE_LINE_SIZE 64

// 캐시 라인 정렬 매크로
#define CACHE_ALIGNED __attribute__((aligned(CACHE_LINE_SIZE)))

// 캐시 친화적 링크드 리스트
struct cache_friendly_node {
void *data;
struct cache_friendly_node *next;
```

```
char padding[CACHE_LINE_SIZE - sizeof(void *) - sizeof(void *)];
} CACHE_ALIGNED;

// 캐시 친화적 해시 테이블
struct cache_friendly_hashtable {
size_t bucket_count;
size_t element_count;
struct cache_friendly_node **buckets CACHE_ALIGNED;
// 통계 정보 (별도 캐시 라인)
struct {
uint64_t lookup_count;
uint64_t collision_count;
uint64_t cache_hits;
uint64_t cache_misses;
} stats CACHE_ALIGNED;
};

// 메모리 프리페칭 최적화
static inline void prefetch_data(const void *addr)
{
#ifdef ARM64
asm volatile("prfm pldl1keep, [%0]" : : "r"(addr));
#elif defined(ARM32)
```

```
asm volatile("pld [%0]" : : "r"(addr));
#endif
}


// 캐시 친화적 배열 순회
static void process_array_cache_friendly(int *array, size_t count)
{
const size_t prefetch_distance = 8; // 8개 요소 앞서 프리페치
for (size_t i = 0; i < count; i++) {
/*
* 미리 프리페치
*/
if (i + prefetch_distance < count) {
prefetch_data(&array[i + prefetch_distance]);
}
/*
* 현재 요소 처리
*/
process_element(array[i]);
}
}
```

2. 암호화 연산 최적화

2.1 하드웨어 가속 활용

```
// ARM Crypto Extension 활용 AES 구현
#ifdef ARM_CE_AES

// AES 라운드 키 확장 (하드웨어 가속)
static void aes_expand_key_hw(const uint8_t *key, uint32_t *round_keys,
int key_bits)
{
int rounds = (key_bits == 128) ? 10 : (key_bits == 192) ? 12 : 14;
/*
* 초기 키 로드
*/
uint8x16_t key_vec = vld1q_u8(key);
vst1q_u8((uint8_t *)round_keys, key_vec);
/*
* 라운드 키 생성 (하드웨어 명령어 사용)
*/
for (int i = 1; i <= rounds; i++) {
uint8_t rcon = aes_rcon_table[i - 1];
// AES 키 확장 명령어
```

```
key_vec = vaeseq_u8(key_vec, vdupq_n_u8(0));

key_vec = vaesmcq_u8(key_vec);

// Rcon 적용

uint8x16_t rcon_vec = vdupq_n_u8(0);

rcon_vec = vsetq_lane_u8(rcon, rcon_vec, 0);

key_vec = veorq_u8(key_vec, rcon_vec);

vst1q_u8((uint8_t *)&round_keys[i * 4], key_vec);

}

}



// AES 블록 암호화 (하드웨어 가속)

static void aes_encrypt_block_hw(const uint8_t *plaintext,

const uint32_t *round_keys,

uint8_t *ciphertext, int rounds)

{

uint8x16_t state = vld1q_u8(plaintext);

/*

* 초기 라운드 키 추가

*/

uint8x16_t round_key = vld1q_u8((const uint8_t *)round_keys);

state = veorq_u8(state, round_key);

/*

* 메인 라운드 (하드웨어 명령어 사용)
```

```
*/
for (int i = 1; i < rounds; i++) {
round_key = vld1q_u8((const uint8_t *)&round_keys[i * 4]);
// AES 암호화 라운드
state = vaeseq_u8(state, round_key);
state = vaesmcq_u8(state);
}
/*
* 최종 라운드
*/
round_key = vld1q_u8((const uint8_t *)&round_keys[rounds * 4]);
state = vaeseq_u8(state, round_key);
vst1q_u8(ciphertext, state);
}


// 병렬 AES 처리 (SIMD 활용)
static void aes_encrypt_parallel(const uint8_t *plaintext,
const uint32_t *round_keys,
uint8_t *ciphertext,
size_t block_count, int rounds)
{
const size_t parallel_blocks = 4; // 4블록 병렬 처리
for (size_t i = 0; i < block_count; i += parallel_blocks) {
```

```
size_t remaining = MIN(parallel_blocks, block_count - i);

/*

* 4개 블록 로드

*/

uint8x16_t states[4];

for (size_t j = 0; j < remaining; j++) {

states[j] = vld1q_u8(&plaintext[(i + j) * 16]);

}

/*

* 병렬 암호화

*/

for (int round = 0; round <= rounds; round++) {

uint8x16_t round_key = vld1q_u8((const uint8_t *)&round_keys[round * 4]);

for (size_t j = 0; j < remaining; j++) {

if (round == 0) {

states[j] = veorq_u8(states[j], round_key);

} else if (round < rounds) {

states[j] = vaeseq_u8(states[j], round_key);

states[j] = vaesmcq_u8(states[j]);

} else {

states[j] = vaeseq_u8(states[j], round_key);

}

}
```

```
}

/*

* 결과 저장

*/

for (size_t j = 0; j < remaining; j++) {

vst1q_u8(&ciphertext[(i + j) * 16], states[j]);

}

}

}


#endif /* ARM_CE_AES */
```

2.2 암호화 컨텍스트 재사용

```
// 암호화 컨텍스트 풀

struct crypto_context_pool {

struct crypto_context *contexts;

bool *in_use;

size_t pool_size;

struct mutex pool_mutex;

uint32_t hit_count;

uint32_t miss_count;

};
```

```
static struct crypto_context_pool aes_pool;
static struct crypto_context_pool rsa_pool;


// 컨텍스트 풀 초기화
static TEE_Result init_crypto_context_pool(struct crypto_context_pool *pool,
size_t pool_size,
crypto_algorithm_t algo)
{
pool->contexts = calloc(pool_size, sizeof(struct crypto_context));
if (!pool->contexts) {
return TEE_ERROR_OUT_OF_MEMORY;
}
pool->in_use = calloc(pool_size, sizeof(bool));
if (!pool->in_use) {
free(pool->contexts);
return TEE_ERROR_OUT_OF_MEMORY;
}
pool->pool_size = pool_size;
pool->hit_count = 0;
pool->miss_count = 0;
mutex_init(&pool->pool_mutex);
/*
```

```
* 컨텍스트 사전 초기화

*/

for (size_t i = 0; i < pool_size; i++) {

crypto_context_init(&pool->contexts[i], algo);

}

return TEE_SUCCESS;

}



// 빠른 컨텍스트 획득

static struct crypto_context *get_crypto_context_fast(

struct crypto_context_pool *pool, const uint8_t *key, size_t key_len)

{

struct crypto_context *ctx = NULL;

mutex_lock(&pool->pool_mutex);

/*

* 1단계: 같은 키를 사용하는 컨텍스트 검색

*/

for (size_t i = 0; i < pool->pool_size; i++) {

if (!pool->in_use[i] &&

pool->contexts[i].key_len == key_len &&

memcmp(pool->contexts[i].key, key, key_len) == 0) {

pool->in_use[i] = true;

ctx = &pool->contexts[i];
```

```
pool->hit_count++;

goto out;

}

}

/*

* 2단계: 빈 컨텍스트 검색

*/

for (size_t i = 0; i < pool->pool_size; i++) {

if (!pool->in_use[i]) {

pool->in_use[i] = true;

ctx = &pool->contexts[i];

/*

* 키 설정

*/

memcpy(ctx->key, key, key_len);

ctx->key_len = key_len;

crypto_context_set_key(ctx, key, key_len);

pool->miss_count++;

goto out;

}

}

/*

* 3단계: 풀이 가득 참 - 새 컨텍스트 생성
```

```
*/
ctx = malloc(sizeof(struct crypto_context));
if (ctx) {
crypto_context_init(ctx, pool->contexts[0].algorithm);
crypto_context_set_key(ctx, key, key_len);
ctx->is_pooled = false;
pool->miss_count++;
}
out:
mutex_unlock(&pool->pool_mutex);
return ctx;
}


// 컨텍스트 반환
static void return_crypto_context(struct crypto_context_pool *pool,
struct crypto_context *ctx)
{
if (!ctx->is_pooled) {
crypto_context_cleanup(ctx);
free(ctx);
return;
}
mutex_lock(&pool->pool_mutex);
```

```
/*
* 풀에서 컨텍스트 인덱스 찾기
*/
size_t index = ctx - pool->contexts;
if (index < pool->pool_size) {
pool->in_use[index] = false;
}
mutex_unlock(&pool->pool_mutex);
}
```

3. 시스템 콜 최적화

3.1 시스템 콜 배치 처리

```
// 배치 시스템 콜 구조체
struct syscall_batch {
uint32_t count;
uint32_t max_count;
struct {
uint32_t syscall_num;
unsigned long args[6];
TEE_Result result;
} calls[MAX_BATCH_SIZE];
```

```
};


// 배치 시스템 콜 실행
TEE_Result syscall_batch_execute(unsigned long batch_addr)
{
struct syscall_batch *batch = NULL;
TEE_Result res = TEE_SUCCESS;
struct ts_session *sess = ts_get_current_session();
/*
* 1단계: 배치 구조체 접근 권한 확인
*/
res = tee_mmu_check_access_rights(to_user_ta_ctx(sess->ctx),
TEE_MEMORY_ACCESS_READ | TEE_MEMORY_ACCESS_WRITE,
batch_addr, sizeof(struct syscall_batch));
if (res != TEE_SUCCESS) {
return res;
}
batch = (struct syscall_batch *)batch_addr;
/*
* 2단계: 배치 크기 검증
*/
if (batch->count > batch->max_count || batch->count > MAX_BATCH_SIZE) {
return TEE_ERROR_BAD_PARAMETERS;
```

```
}

/*

* 3단계: 배치 내 시스템 콜들 순차 실행

*/

for (uint32_t i = 0; i < batch->count; i++) {

syscall_t scf = get_tee_syscall_func(batch->calls[i].syscall_num);

/*

* 개별 시스템 콜 실행

*/

batch->calls[i].result = scf(batch->calls[i].args[0],

batch->calls[i].args[1],

batch->calls[i].args[2],

batch->calls[i].args[3],

batch->calls[i].args[4],

batch->calls[i].args[5]);

/*

* 에러 발생 시 조기 종료 (선택적)

*/

if (batch->calls[i].result != TEE_SUCCESS &&

batch->calls[i].syscall_num != TEE_SCN_LOG) {

// 로그는 실패해도 계속 진행

break;

}
```

```
}

return TEE_SUCCESS;

}



// 클라이언트 측 배치 API

static TEE_Result batch_crypto_operations(TEE_OperationHandle *operations,

const uint8_t **inputs,

uint8_t **outputs,

const size_t *input_lens,

size_t *output_lens,

size_t count)

{

struct syscall_batch batch = {0};

batch.max_count = MAX_BATCH_SIZE;

batch.count = 0;

/*

* 배치에 암호화 연산들 추가

*/

for (size_t i = 0; i < count && batch.count < MAX_BATCH_SIZE; i++) {

batch.calls[batch.count].syscall_num = TEE_SCN_CIPHER_DO_FINAL;

batch.calls[batch.count].args[0] = (unsigned long)operations[i];

batch.calls[batch.count].args[1] = (unsigned long)inputs[i];

batch.calls[batch.count].args[2] = input_lens[i];
```

```
batch.calls[batch.count].args[3] = (unsigned long)outputs[i];

batch.calls[batch.count].args[4] = (unsigned long)&output_lens[i];

batch.calls[batch.count].args[5] = 0;

batch.count++;

}

/*

* 배치 실행

*/

return syscall_batch_execute((unsigned long)&batch);

}
```

3.2 시스템 콜 캐싱

```
// 시스템 콜 결과 캐시

struct syscall_cache_entry {

uint32_t syscall_num;

unsigned long args[6];

TEE_Result result;

uint64_t timestamp;

bool valid;

};


static struct syscall_cache_entry syscall_cache[SYSCALL_CACHE_SIZE];
```

```
static struct mutex syscall_cache_mutex = MUTEX_INITIALIZER;


// 캐시 키 생성
static uint32_t generate_cache_key(uint32_t syscall_num, unsigned long *args)
{
uint32_t hash = syscall_num;
/*
* 간단한 해시 함수 (FNV-1a 변형)
*/
for (int i = 0; i < 6; i++) {
hash ^= (uint32_t)(args[i] & 0xFFFFFFFF);
hash *= 0x01000193;
hash ^= (uint32_t)(args[i] >> 32);
hash *= 0x01000193;
}
return hash % SYSCALL_CACHE_SIZE;
}


// 캐시에서 결과 검색
static bool lookup_syscall_cache(uint32_t syscall_num, unsigned long *args,
TEE_Result *result)
{
uint32_t key = generate_cache_key(syscall_num, args);
```

```
bool found = false;

uint64_t current_time = get_time_us();

mutex_lock(&syscall_cache_mutex);

struct syscall_cache_entry *entry = &syscall_cache[key];

/*

* 캐시 엔트리 유효성 검사

*/

if (entry->valid &&

entry->syscall_num == syscall_num &&

(current_time - entry->timestamp) < CACHE_TIMEOUT_US) {

/*

* 인자 비교

*/

bool args_match = true;

for (int i = 0; i < 6; i++) {

if (entry->args[i] != args[i]) {

args_match = false;

break;

}

}

if (args_match) {

*result = entry->result;

found = true;
```

```
    }

    }

    mutex_unlock(&syscall_cache_mutex);

    return found;

    }



    // 캐시에 결과 저장

    static void store_syscall_cache(uint32_t syscall_num, unsigned long *args,

    TEE_Result result)

    {

    uint32_t key = generate_cache_key(syscall_num, args);

    uint64_t current_time = get_time_us();

    mutex_lock(&syscall_cache_mutex);

    struct syscall_cache_entry *entry = &syscall_cache[key];

    entry->syscall_num = syscall_num;

    memcpy(entry->args, args, sizeof(entry->args));

    entry->result = result;

    entry->timestamp = current_time;

    entry->valid = true;

    mutex_unlock(&syscall_cache_mutex);

    }



    // 캐시 가능한 시스템 콜 확인
```

```
static bool is_cacheable_syscall(uint32_t syscall_num)

{

/*

* 읽기 전용이고 부작용이 없는 시스템 콜만 캐시

*/

switch (syscall_num) {

case TEE_SCN_GET_PROPERTY:

case TEE_SCN_GET_PROPERTY_NAME_TO_INDEX:

case TEE_SCN_GET_TIME:

return true;

default:

return false;

}

}
```

4. 스레드 및 동시성 최적화

4.1 락-프리 데이터 구조

```
// 락-프리 큐 구현

struct lockfree_queue {

volatile struct queue_node *head;

volatile struct queue_node *tail;
```

```
char padding1[CACHE_LINE_SIZE - 2 * sizeof(void *)];

// 통계 정보 (별도 캐시 라인)

volatile uint64_t enqueue_count;

volatile uint64_t dequeue_count;

char padding2[CACHE_LINE_SIZE - 2 * sizeof(uint64_t)];

};


struct queue_node {

volatile void *data;

volatile struct queue_node *next;

};


// 락-프리 큐 초기화

static void lockfree_queue_init(struct lockfree_queue *queue)

{

struct queue_node *dummy = malloc(sizeof(struct queue_node));

dummy->data = NULL;

dummy->next = NULL;

queue->head = dummy;

queue->tail = dummy;

queue->enqueue_count = 0;

queue->dequeue_count = 0;

}
```

```
// 락-프리 enqueue
static bool lockfree_enqueue(struct lockfree_queue *queue, void *data)
{
struct queue_node *new_node = malloc(sizeof(struct queue_node));
if (!new_node) {
return false;
}
new_node->data = data;
new_node->next = NULL;
while (true) {
struct queue_node *tail = (struct queue_node *)queue->tail;
struct queue_node *next = (struct queue_node *)tail->next;
/*
* tail이 여전히 마지막 노드인지 확인
*/
if (tail == queue->tail) {
if (next == NULL) {
/*
* tail->next를 새 노드로 설정 시도
*/
if (__atomic_compare_exchange_n(&tail->next, &next, new_node,
false, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST)) {
```

```
/*
* tail을 새 노드로 이동
*/
__atomic_compare_exchange_n(&queue->tail, &tail, new_node,
false, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST);
break;
}
} else {
/*
* tail을 다음 노드로 이동 시도
*/
__atomic_compare_exchange_n(&queue->tail, &tail, next,
false, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST);
}
}
}
__atomic_fetch_add(&queue->enqueue_count, 1, __ATOMIC_SEQ_CST);
return true;
}


// 락-프리 dequeue
static void *lockfree_dequeue(struct lockfree_queue *queue)
{
```

```
while (true) {
struct queue_node *head = (struct queue_node *)queue->head;
struct queue_node *tail = (struct queue_node *)queue->tail;
struct queue_node *next = (struct queue_node *)head->next;
/*
* head가 여전히 첫 번째 노드인지 확인
*/
if (head == queue->head) {
if (head == tail) {
if (next == NULL) {
/*
* 큐가 비어있음
*/
return NULL;
} else {
/*
* tail을 다음 노드로 이동 시도
*/
__atomic_compare_exchange_n(&queue->tail, &tail, next,
false, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST);
}
} else {
if (next == NULL) {
```

```
/*
* 일관성 없는 상태, 재시도
*/
continue;
}
/*
* 데이터 읽기
*/
void *data = (void *)next->data;
/*
* head를 다음 노드로 이동
*/
if (__atomic_compare_exchange_n(&queue->head, &head, next,
false, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST)) {
free(head);
__atomic_fetch_add(&queue->dequeue_count, 1, __ATOMIC_SEQ_CST);
return data;
}
}
}
}
}
```

4.2 작업 스틸링 스케줄러

```
// 작업 스틸링 큐
struct work_stealing_queue {
volatile void **buffer;
volatile size_t capacity;
volatile size_t mask;
// 로컬 스레드만 접근 (캐시 라인 분리)
volatile size_t head CACHE_ALIGNED;
// 다른 스레드들이 접근 (캐시 라인 분리)
volatile size_t tail CACHE_ALIGNED;
};

// 작업 스틸링 스케줄러
struct work_stealing_scheduler {
struct work_stealing_queue *queues;
size_t num_threads;
volatile bool shutdown;
// 전역 통계
volatile uint64_t total_tasks;
volatile uint64_t stolen_tasks;
};
```

```
// 로컬 큐에 작업 추가 (빠른 경로)

static bool ws_queue_push_local(struct work_stealing_queue *queue, void *task)

{

size_t head = queue->head;

size_t next_head = (head + 1) & queue->mask;

/*

* 큐가 가득 찬지 확인

*/

if (next_head == queue->tail) {

return false; // 큐 가득 찬

}

/*

* 작업 추가

*/

queue->buffer[head] = task;

/*

* 메모리 배리어 후 head 업데이트

*/

__atomic_store_n(&queue->head, next_head, __ATOMIC_RELEASE);

return true;

}



// 로컬 큐에서 작업 제거 (빠른 경로)
```

```
static void *ws_queue_pop_local(struct work_stealing_queue *queue)

{

size_t head = queue->head;

if (head == queue->tail) {

return NULL; // 큐 비어있음

}

/*

* head를 먼저 감소

*/

head = (head - 1) & queue->mask;

__atomic_store_n(&queue->head, head, __ATOMIC_RELAXED);

/*

* 작업 읽기

*/

void *task = (void *)queue->buffer[head];

/*

* 다른 스레드가 스틸했는지 확인

*/

size_t tail = __atomic_load_n(&queue->tail, __ATOMIC_ACQUIRE);

if (head > tail) {

/*

* 성공적으로 작업 획득

*/
```

```
return task;

}

if (head == tail) {

/*

* 마지막 작업 - 경쟁 상황

*/

size_t new_tail = tail + 1;

if (__atomic_compare_exchange_n(&queue->tail, &tail, new_tail,

false, __ATOMIC_SEQ_CST, __ATOMIC_RELAXED)) {

/*

* 성공적으로 작업 획득

*/

return task;

}

}

/*

* 실패 - head 복원

*/

__atomic_store_n(&queue->head, (head + 1) & queue->mask, __ATOMIC_RELAXED);

return NULL;

}



// 다른 큐에서 작업 스틸
```

```
static void *ws_queue_steal(struct work_stealing_queue *queue)

{

size_t tail = __atomic_load_n(&queue->tail, __ATOMIC_ACQUIRE);

size_t head = __atomic_load_n(&queue->head, __ATOMIC_ACQUIRE);

if (tail >= head) {

return NULL; // 큐 비어있음

}

/*

* 작업 읽기

*/

void *task = (void *)queue->buffer[tail];

/*

* tail 증가 시도

*/

if (__atomic_compare_exchange_n(&queue->tail, &tail, tail + 1,

false, __ATOMIC_SEQ_CST, __ATOMIC_RELAXED)) {

return task;

}

return NULL; // 다른 스레드가 먼저 스틸함

}



// 작업자 스레드 메인 루프

static void worker_thread_main(struct work_stealing_scheduler *scheduler,
```

```
int thread_id)

{

struct work_stealing_queue *local_queue = &scheduler->queues[thread_id];

while (!scheduler->shutdown) {

void *task = NULL;

/*

* 1단계: 로컬 큐에서 작업 시도

*/

task = ws_queue_pop_local(local_queue);

if (!task) {

/*

* 2단계: 다른 큐들에서 작업 스틸 시도

*/

for (size_t i = 0; i < scheduler->num_threads; i++) {

if (i != thread_id) {

task = ws_queue_steal(&scheduler->queues[i]);

if (task) {

__atomic_fetch_add(&scheduler->stolen_tasks, 1,

__ATOMIC_RELAXED);

break;

}

}

}
```

```
}

if (task) {

/*

* 작업 실행

*/

execute_task(task);

__atomic_fetch_add(&scheduler->total_tasks, 1, __ATOMIC_RELAXED);

} else {

/*

* 작업이 없으면 잠시 대기

*/

thread_yield();

}

}

}
```

5. I/O 및 스토리지 최적화

5.1 비동기 I/O 처리

```
// 비동기 I/O 요청 구조체

struct async_io_request {

enum io_operation_type {
```

```
IO_READ,

IO_WRITE,

IO_SYNC

} operation;

void *buffer;

size_t size;

off_t offset;

// 콜백 정보

void (*completion_callback)(struct async_io_request *req, TEE_Result result);

void *callback_data;

// 상태 정보

volatile enum {

IO_PENDING,

IO_IN_PROGRESS,

IO_COMPLETED,

IO_ERROR

} status;

TEE_Result result;

uint64_t submit_time;

uint64_t completion_time;

struct async_io_request *next;

};
```

```
// 비동기 I/O 큐
struct async_io_queue {
struct async_io_request *head;
struct async_io_request *tail;
struct mutex queue_mutex;
struct condvar queue_cv;
// 통계
uint64_t submitted_requests;
uint64_t completed_requests;
uint64_t total_latency;
};

static struct async_io_queue io_queue = {0};

// 비동기 I/O 요청 제출
static TEE_Result submit_async_io(struct async_io_request *req)
{
req->status = IO_PENDING;
req->submit_time = get_time_us();
mutex_lock(&io_queue.queue_mutex);
/*
* 큐에 요청 추가
*/
```

```
req->next = NULL;

if (io_queue.tail) {

io_queue.tail->next = req;

} else {

io_queue.head = req;

}

io_queue.tail = req;

io_queue.submitted_requests++;

/*

* I/O 스레드 깨우기

*/

condvar_signal(&io_queue.queue_cv);

mutex_unlock(&io_queue.queue_mutex);

return TEE_SUCCESS;

}



// I/O 처리 스레드

static void io_worker_thread(void)

{

while (true) {

struct async_io_request *req = NULL;

mutex_lock(&io_queue.queue_mutex);

/*
```

```
* 요청 대기
*/
while (!io_queue.head) {
condvar_wait(&io_queue.queue_cv, &io_queue.queue_mutex);
}
/*
* 요청 제거
*/
req = io_queue.head;
io_queue.head = req->next;
if (!io_queue.head) {
io_queue.tail = NULL;
}
mutex_unlock(&io_queue.queue_mutex);
/*
* I/O 작업 수행
*/
req->status = IO_IN_PROGRESS;
switch (req->operation) {
case IO_READ:
req->result = perform_read_operation(req->buffer, req->size, req->offset);
break;
case IO_WRITE:
```

```
req->result = perform_write_operation(req->buffer, req->size, req->offset);

break;

case IO_SYNC:

req->result = perform_sync_operation();

break;

}

/*

* 완료 처리

*/

req->completion_time = get_time_us();

req->status = (req->result == TEE_SUCCESS) ? IO_COMPLETED : IO_ERROR;

/*

* 콜백 호출

*/

if (req->completion_callback) {

req->completion_callback(req, req->result);

}

/*

* 통계 업데이트

*/

mutex_lock(&io_queue.queue_mutex);

io_queue.completed_requests++;

io_queue.total_latency += (req->completion_time - req->submit_time);
```

```
mutex_unlock(&io_queue.queue_mutex);

}

}
```

5.2 스토리지 캐싱 최적화

```
// LRU 캐시 구현

struct lru_cache_node {

void *key;

size_t key_size;

void *data;

size_t data_size;

struct lru_cache_node *prev;

struct lru_cache_node *next;

struct lru_cache_node *hash_next;

uint64_t access_time;

uint32_t access_count;

};


struct lru_cache {

struct lru_cache_node **hash_table;

size_t hash_table_size;

struct lru_cache_node *head; // 가장 최근 사용
```

```
struct lru_cache_node *tail; // 가장 오래된 사용

size_t current_size;

size_t max_size;

size_t max_entries;

struct mutex cache_mutex;

// 통계

uint64_t hit_count;

uint64_t miss_count;

uint64_t eviction_count;

};



// 캐시 해시 함수

static uint32_t cache_hash(const void *key, size_t key_size, size_t table_size)

{

const uint8_t *bytes = (const uint8_t *)key;

uint32_t hash = 2166136261U; // FNV offset basis

for (size_t i = 0; i < key_size; i++) {

hash ^= bytes[i];

hash *= 16777619U; // FNV prime

}

return hash % table_size;

}
```

```
// 캐시에서 데이터 검색

static void *lru_cache_get(struct lru_cache *cache, const void *key,

size_t key_size, size_t *data_size)

{

uint32_t hash = cache_hash(key, key_size, cache->hash_table_size);

void *result = NULL;

mutex_lock(&cache->cache_mutex);

/*

* 해시 테이블에서 검색

*/

struct lru_cache_node *node = cache->hash_table[hash];

while (node) {

if (node->key_size == key_size &&

memcmp(node->key, key, key_size) == 0) {

/*

* 캐시 히트 - 노드를 head로 이동

*/

if (node != cache->head) {

// 현재 위치에서 제거

if (node->prev) {

node->prev->next = node->next;

}

if (node->next) {
```

```
node->next->prev = node->prev;

}

if (node == cache->tail) {

cache->tail = node->prev;

}

// head에 삽입

node->prev = NULL;

node->next = cache->head;

if (cache->head) {

cache->head->prev = node;

}

cache->head = node;

if (!cache->tail) {

cache->tail = node;

}

}

/*

* 접근 통계 업데이트

*/

node->access_time = get_time_us();

node->access_count++;

/*

* 데이터 복사
```

```
*/
result = malloc(node->data_size);
if (result) {
memcpy(result, node->data, node->data_size);
*data_size = node->data_size;
}
cache->hit_count++;
break;
}
node = node->hash_next;
}
if (!result) {
cache->miss_count++;
}
mutex_unlock(&cache->cache_mutex);
return result;
}


// 캐시에 데이터 저장
static TEE_Result lru_cache_put(struct lru_cache *cache, const void *key,
size_t key_size, const void *data, size_t data_size)
{
uint32_t hash = cache_hash(key, key_size, cache->hash_table_size);
```

```
struct lru_cache_node *node = NULL;

/*

* 새 노드 할당

*/

node = malloc(sizeof(struct lru_cache_node));

if (!node) {

return TEE_ERROR_OUT_OF_MEMORY;

}

node->key = malloc(key_size);

node->data = malloc(data_size);

if (!node->key || !node->data) {

free(node->key);

free(node->data);

free(node);

return TEE_ERROR_OUT_OF_MEMORY;

}

/*

* 노드 초기화

*/

memcpy(node->key, key, key_size);

memcpy(node->data, data, data_size);

node->key_size = key_size;

node->data_size = data_size;
```

```
node->access_time = get_time_us();

node->access_count = 1;

mutex_lock(&cache->cache_mutex);

/*

* 캐시 크기 제한 확인 및 eviction

*/

while (cache->current_size + data_size > cache->max_size && cache->tail) {

struct lru_cache_node *victim = cache->tail;

/*

* tail에서 제거

*/

cache->tail = victim->prev;

if (cache->tail) {

cache->tail->next = NULL;

} else {

cache->head = NULL;

}

/*

* 해시 테이블에서 제거

*/

uint32_t victim_hash = cache_hash(victim->key, victim->key_size,

cache->hash_table_size);

struct lru_cache_node **hash_ptr = &cache->hash_table[victim_hash];
```

```
while (*hash_ptr && *hash_ptr != victim) {
hash_ptr = &(*hash_ptr)->hash_next;
}
if (*hash_ptr) {
*hash_ptr = victim->hash_next;
}
/*
* 메모리 해제
*/
cache->current_size -= victim->data_size;
free(victim->key);
free(victim->data);
free(victim);
cache->eviction_count++;
}
/*
* 새 노드를 head에 삽입
*/
node->prev = NULL;
node->next = cache->head;
if (cache->head) {
cache->head->prev = node;
}
```

```
cache->head = node;

if (!cache->tail) {

cache->tail = node;

}

/*

* 해시 테이블에 추가

*/

node->hash_next = cache->hash_table[hash];

cache->hash_table[hash] = node;

cache->current_size += data_size;

mutex_unlock(&cache->cache_mutex);

return TEE_SUCCESS;

}
```

이 성능 최적화 실전 기법 문서는 OP-TEE 시스템의 다양한 측면에서 성능을 향상시킬 수 있는 구체적인 방법들을 제공합니다. 각 기법은 실제 코드 예제와 함께 설명되어 있어 바로 적용할 수 있습니다.